UNIVERSIDADE FEDERAL DA FRONTEIRA SUL (UFFS)
CAMPUS CHAPECÓ – CURSO DE CIÊNCIA DA COMPUTAÇÃO
CCR: LINGUAGENS FORMAIS E AUTÔMATOS — Prof. Braulio Mello

Construção e Validação Linear de Autômato Finito Determinístico Mínimo com Estado de Erro a partir de Especificações Léxicas e Gramáticas Regulares

Relatório Técnico-Científico de Concepção, Algoritmos e Implementação em C++

Acadêmicos do CCR de Linguagens Formais e Autômatos
Matheus Lazarotto, Luiz Augusto Scarsi
Chapecó – SC, Novembro de 2026

Resumo — Este trabalho apresenta o projeto, a especificação formal e a implementação em linguagem C++17 de uma ferramenta computacional para conversão integrada de palavras reservadas (tokens literais) e Gramáticas Regulares (GR em notação BNF) em um Autômato Finito Determinístico (AFD) mínimo, provido de estado de erro e estritamente livre de estados inalcançáveis e mortos, sem o emprego de partição por classes de equivalência. A arquitetura do sistema foi estruturada em um pipeline linear de cinco etapas sequenciais: (1) instanciação do Autômato Finito Não-Determinístico (AFND) com unicidade estrita do estado inicial S e caminhos exclusivos para cada token; (2) determinização clássica por construção de subconjuntos com cálculo de fecho-ε; (3) minimização estrutural sem classes de equivalência por eliminação de estados inalcançáveis (busca em largura direta) e estados mortos (busca em largura reversa no grafo transposto); (4) totalização da função de transição pela anexação final de um estado de erro sumidouro dotado de auto-laços reflexivos; e (5) verificação estrutural e simulação de reconhecimento de sentenças em tempo linear O(n) com rastreamento completo de transições e tokens identificados. A validação experimental sobre o caso base do CCR demonstrou a progressão quantitativa de estados (14 -> 12 -> 12 -> 13), comprovando a robustez dos algoritmos e a aderência integral às diretrizes curriculares estabelecidas.

Palavras-chave: Linguagens Formais; Autômatos Finitos Determinísticos; Gramáticas Regulares; Determinização por Subconjuntos; Minimização Estrutural; Estado de Erro; Análise Léxica.

Abstract — This paper presents the formal design and C++17 implementation of a computational tool for transforming reserved words (literal tokens) and Regular Grammars (RG in BNF notation) into a minimal Deterministic Finite Automaton (DFA) equipped with a sink error state and strictly free of unreachable and dead states, without applying equivalence class partitioning. The architecture is structured into a linear five-stage pipeline: (1) instantiation of a Non-Deterministic Finite Automaton (NFA) with a single shared initial state S and exclusive paths for each token; (2) classical subset construction determinization with epsilon-closure evaluation; (3) structural minimization without equivalence classes by eliminating unreachable states (forward BFS) and dead states (reverse BFS on the transposed graph); (4) transition function totalization by appending a final sink error state; and (5) structural verification and linear-time O(n) sentence recognition with full path tracing. Experimental verification on benchmark instances confirmed the state progression (14 -> 12 -> 12 -> 13), demonstrating correctness and total compliance with formal curriculum requirements.

Keywords: Formal Languages; Deterministic Finite Automata; Regular Grammars; Subset Construction; Structural Minimization; Error State; Lexical Analysis.

1. Introdução

Na arquitetura clássica de processadores de linguagens e compiladores, a análise léxica (ou scanner) atua como a etapa inicial do front-end, responsável pela leitura do fluxo contínuo de caracteres de entrada e pelo seu agrupamento em unidades léxicas elementares denominadas tokens. Enquanto as palavras reservadas e os padrões sintáticos de identificadores e numerais são naturalmente especificados por regras de Gramáticas Regulares (GR) lineares à direita ou listas de sentenças literais, sua transcrição direta para máquinas de estado finito resulta habitualmente em Autômatos Finitos Não-Determinísticos (AFND).

Embora o modelo não-determinístico possua o mesmo poder expressivo do modelo determinístico para a classe das linguagens regulares, sua execução em computadores digitais convencionais exigiria algoritmos de retrocesso (backtracking) ou paralelismo simulado, implicando custos proibitivos de processamento. Torna-se mandatório, portanto, converter o AFND em um Autômato Finito Determinístico (AFD) equivalente, no qual a leitura de cada

caractere determine um estado de destino unívoco, assegurando tempo de reconhecimento estritamente linear em função do comprimento da cadeia de entrada (O(n)).

No âmbito pedagógico e técnico do CCR de Linguagens Formais e Autômatos da UFFS, a especificação curricular estabelece três diretrizes estruturais de projeto:
• Minimização Estrutural Específica: O autômato gerado deve ser mínimo estritamente sob o critério de ausência de estados inalcançáveis e mortos, vetando expressamente o emprego de particionamento por classes de equivalência entre estados indistinguíveis;
• Função de Transição Total: O autômato deve ser completo, de modo que qualquer combinação de estado e símbolo não prevista pela linguagem seja direcionada para um estado de erro (sumidouro ou sink);
• Ordem Rígida de Execução: O estado de erro deve ser anexado ao final do processo de minimização, evitando distorções conceituais, uma vez que o estado de erro, por definição formal, é incapaz de alcançar qualquer estado final de aceitação.

Este trabalho tem como objetivo descrever a formalização teórica, a metodologia linear e a implementação desenvolvida em C++17, demonstrando passo a passo as etapas de transformação e a validação experimental completa da ferramenta sobre os casos de teste especificados.

2. Fundamentação Teórica e Modelo Formal

2.1 Gramáticas Regulares e Sentenças Literais: Uma Gramática Regular à direita (tipo 3 na Hierarquia de Chomsky) é formalmente uma quádrupla G = (V_N, V_T, P, S), onde V_N representa o conjunto finito de variáveis não-terminais, V_T o conjunto finito de símbolos terminais, S o símbolo inicial e P as produções restritas às formas A -> aB, A -> a ou A -> ε. Tokens literais correspondem a palavras finitas tratadas isomorphicamente como cadeias lineares estritas de derivação.

2.2 Autômato Finito Não-Determinístico (AFND): O AFND gerado na primeira etapa é definido pela 5-upla M = (Q, Σ, δ, S, F), onde Q é o conjunto de estados, Σ o alfabeto, δ a função de transição com múltiplos destinos, S o estado inicial compartilhado e F o conjunto de estados aceitadores.

2.3 Teorema da Determinização por Construção de Subconjuntos: Todo AFND possui um AFD equivalente onde cada estado D_i representa um subconjunto de estados do AFND original. A transição determinística é dada por δ_AFD(D_i, a) = fecho-ε(∪_{q ∈ D_i} δ(q, a)), e um estado D_i é aceitador se contiver ao menos um estado aceitador do AFND.

2.4 Minimização Estrutural sem Classes de Equivalência: A minimização consiste estritamente na remoção de duas anomalias estruturais:
• Estados Inalcançáveis: Nós q ∈ Q_AFD inacessíveis a partir do estado inicial (detectados em tempo linear por BFS direta a partir de q_0);
• Estados Mortos: Nós q ∈ Q_AFD incapazes de atingir qualquer estado final de aceitação (detectados em tempo linear por BFS reversa no grafo transposto G^T a partir dos estados finais F).

2.5 Totalidade da Função de Transição via Estado de Erro: Para satisfazer a condição de função total δ: Q × Σ -> Q, anexa-se um nó terminal sumidouro q_erro dotado de auto-laços reflexivos para todo o alfabeto. Todas as células indefinidas da matriz de transição convergem para q_erro.

3. Metodologia e Pipeline Linear da Aplicação

A arquitetura do software organiza-se modularmente em C++17 (pastas include/ e src/), dividindo o processamento em um pipeline linear de cinco etapas sequenciais:

3.1 Modelo de Dados Central (include/Automato.h e src/Automato.cpp): Encapsula estados, transições, alfabeto dinâmico e estado de erro. O identificador numérico de cada nó coincide com seu índice vetorial (acesso em O(1)). O método imprimirTabela() exibe a matriz de transições formatada com alinhamento tabular e legendas para estado inicial (->), final (*) e erro (E).

3.2 Etapa 1: Carga e Construção do AFND (src/LeitorArquivo.cpp): Realiza o parsing simultâneo de tokens e gramáticas BNF. Unicidade estrita do estado inicial S; cada token recebe caminhos privativos com estados novos,

gerando indeterminismo natural em S (ex: 'se' e 'senao' possuem transições distintas sob 's'). Para a GR, o não-terminal inicial vincula-se a S e alternativas vazias (ε) marcam o estado como aceitador ('variavel').

3.3 Etapa 2: Determinização Algorítmica (src/Determinizar.cpp): Emprega construção de subconjuntos com std::set<int>, tabela de busca std::map<std::set<int>, int> e fila std::queue. Herda cumulativamente os nomes de tokens dos estados componentes.

3.4 Etapa 3: Minimização Estrutural de Inúteis (src/Minimizar.cpp): Realiza a poda de estados inalcançáveis por busca em largura para frente e de estados mortos por busca em largura retrógrada no grafo com arestas invertidas. Os estados sobreviventes são renumerados de modo contíguo (M0, M1, ...).

3.5 Etapa 4: Adição do Estado de Erro e Matriz Total (src/CompletarAutomato.cpp): Inserido obrigatoriamente após a minimização para evitar que o próprio estado de erro seja eliminado como morto. Preenche todas as células vazias da tabela e estabelece laços reflexivos completos.

3.6 Etapa 5: Diagnóstico e Reconhecimento com Rastreio (src/Reconhecer.cpp e src/VerificarEstados.cpp): O motor de reconhecimento avalia sentenças caractere a caractere em O(n), imprimindo o caminho percorrido e os tokens associados. O módulo VerificarEstados audita a vivacidade e alcance da máquina final.

4. Estudo de Caso Experimental e Validação dos Resultados

A validação empírica teve como base o arquivo de teste fornecido pelo docente (exemplos/professor.txt), composto pelos tokens 'se', 'entao', 'senao' e a Gramática Regular de vogais (<S> ::= a<A>|e<A>|i<A>|o<A>|u<A> e <A> ::= a<A>|e<A>|i<A>|o<A>|u<A>|ε).

4.1 Evolução Sequencial de Estados no Pipeline:
• AFND Inicial (Etapa 1): 14 estados (S, A..M). Indeterminismo nos símbolos 's' (destinos A, H) e 'e' (destinos C, M).
• AFD Determinado (Etapa 2): 12 estados determinizados (D0 a D11). Resolução de subconjuntos com herança de tokens (D1={M} -> variavel, D2={C,M} -> variavel, D5={B,I} -> se, D10={G} -> entao, D11={L} -> senao).
• AFD Minimizado (Etapa 3): 12 estados mantidos (M0 a M11). Diagnosticou 0 inalcançáveis e 0 mortos, confirmando que o AFD gerado já era enxuto.
• AFD Final Completo (Etapa 4): 13 estados (M0 a M11 acrescidos de Erro). Preenchimento exato de 72 transições ausentes direcionadas ao Erro e 8 auto-laços reflexivos.

4.2 Validação de Sentenças no Reconhecedor:
• 'se': M0 --s--> M3 --e--> M5 [ACEITA: se]
• 'entao': M0 --e--> M2 --n--> M4 --t--> M6 --a--> M8 --o--> M10 [ACEITA: entao]
• 'senao': M0 --s--> M3 --e--> M5 --n--> M7 --a--> M9 --o--> M11 [ACEITA: senao]
• 'e': M0 --e--> M2 [ACEITA: variavel] (D2={C,M} herda a aceitação da vogal M)
• 'aeiou': M0 --a--> M1 --e--> M1 --i--> M1 --o--> M1 --u--> M1 [ACEITA: variavel]
• 'sen': M0 --s--> M3 --e--> M5 --n--> M7 [REJEITADA: estado M7 não-final]
• 'st': M0 --s--> M3 --t--> Erro [REJEITADA: transição direcionada ao estado de erro]
• 'xyz': [REJEITADA: símbolo 'x' não pertence ao alfabeto]

4.3 Validação Estrutural com Inúteis (exemplos/exemplo3_inalcancavel_e_morto.txt):
Submissão de regras anômalas com laço estéril <M> ::= m<M> e símbolo desconectado <Z> ::= z<Z>|ε. A Etapa 3 detectou e expurgou com exatidão o estado morto e a componente inalcançável, confirmando a robustez da poda de grafos.

5. Diretrizes para Arguição Oral e Decisões de Projeto

1. Reconhecimento da cadeia 'e' como variável: No AFND inicial, 'e' transita simultaneamente para o início de 'entao' (estado C) e para a gramática de vogais (estado aceitador M). A determinização gera o subconjunto D2 = {C, M}, que herda a condição final de M e valida 'e' como variável com correção formal.
2. Precedência da Minimização sobre o Estado de Erro: O estado de erro é um sorvedouro que jamais alcança um estado final. Se inserido antes da minimização, a busca em largura reversa o classificaria e eliminaria sumariamente como estado morto.
3. Minimização sem Classes de Equivalência: Ao contrário de técnicas que particionam estados em blocos indistinguíveis, o método implementado opera como poda topológica de grafo, expurgando unicamente componentes inalcançáveis e mortas.
4. Eficiência Linear da Detecção de Mortos: Ao inverter as arestas (grafo transposto G^T) e disparar uma BFS única a partir dos estados finais, obtém-se o conjunto de estados vivos em tempo O(|V| + |E|).

6. Conclusão

A aplicação computacional desenvolvida atendeu com fidelidade e rigor científico a todos os requisitos formulados para o CCR de Linguagens Formais e Autômatos da UFFS. O fluxo linear de dados assegurou total transparência em cada estágio de processamento, consolidando uma ferramenta computacional robusta para o estudo prático dos mecanismos de análise léxica em compiladores.

7. Referências Bibliográficas

[1] MENEZES, P. B. Linguagens Formais e Autômatos. 6. ed. Porto Alegre: Bookman, 2011.

[2] AHO, A. V.; LAM, M. S.; SETHI, R.; ULLMAN, J. D. Compiladores: Princípios, Técnicas e Ferramentas. 2. ed. São Paulo: Pearson/Prentice Hall, 2008.

[3] DIVERIO, T. A.; MENEZES, P. B. Teoria da Computação: Máquinas Universais e Computabilidade. 3. ed. Porto Alegre: Bookman, 2011.

[4] DAVIS, M. D.; SIGAL, Ron; WEYUKER, E. J. Computability, Complexity, and Languages: Fundamentals of Theoretical Computer Science. 2. ed. Boston: Academic Press, Harcourt, Brace, 1994.

[5] FURTADO, O. J. V. Apostila de Linguagens Formais e Compiladores. [Florianópolis]: UFSC, 2002. Disponível em: www.inf.ufsc.br/~olinto/apostila-lfc.doc.

[6] HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. Introdução à Teoria dos Autômatos, Linguagens e Computação. Rio de Janeiro: Campus, 2003.